Agent Task Contract

University Student-Support Case Agent

Group	Group M Evening
Course	BSE4104 Emerging Trends in Software Engineering
Date	29 September 2026
Repository	https://github.com/SumayyaN/BSE41014_capstone_group_m_eve

1. Goal

The agent helps a **Makerere University freshman** (first-year student) with **onboarding** questions-webmail, campus Wi-Fi, MUELE, registration requirements, and similar topics covered in the approved document corpus.

For each student message, the agent pursues this goal in order:

1. **Ground** the answer in approved documents when possible (retrieve, cite sources).
2. **Escalate** only when retrieval does not support an answer, the student still needs help, or the student explicitly asks for staff follow-up by drafting a **local support case** for a support-desk staff member to review.

Students using this chat are treated as freshmen. The agent does not require them to say so. It refuses the turn when a student says they are a continuing, returning, or postgraduate student.

The agent is **not** a decision-maker: it retrieves, explains, and drafts. Staff remain responsible for resolving cases and for any admissions, grading, disciplinary, or fee **decisions**.

2. Tools

The agent has **exactly two** tools. The **orchestration layer** (application code) maintains an allow-list: the model may *request* a tool, but only the orchestration layer **executes** it after checks in Sections 4 and 5.

Tool 1: `retrieve_documents`

Purpose. Search the Makerere freshman onboarding corpus and return passages that meet the similarity threshold.

Input (JSON)

Field	Type	Required	Description
`query`	string	yes	The student's question or a shorter search query.
`top_k`	integer	no	Number of passages to return. Default 3. Maximum 3.

Output (JSON)

Field	Type	Description
`status`	string	`ok`, `no_match`, or `error`.
`chunks`	array	On `ok`: objects with `chunk_id`, `text`, `source_title`, `source_url`, `similarity_score`. Otherwise empty.
`retrieved_count`	integer	Number of chunks returned.
`message`	string	Present on `error` (e.g. missing query, blocked topic, index unavailable).

Behaviour

- Read-only access to the local index `data/chunks/chunks_with_embeddings.json`.
- Similarity threshold **0.70** (same as `src/rag/retrieve.py`). Chunks below threshold are not returned; if none qualify, `status` is `no_match`.
- Query embedding uses the configured embedding API (network call for the query only). The corpus file is not modified.
- Questions matching **blocked topics** (admissions, grading, disciplinary)—see Section 4—return `status: error` with an appropriate message; no chunks are returned.

On failure

- Missing or empty `query` -> `error`.
- Missing or unreadable index -> `error` (do not invent document text).
- Embedding or retrieval exception -> `error`.

Tool 2: `create_support_case`

Purpose. Create a **draft** support case record for staff review. The tool does **not** email offices, change live university systems, resolve, close, or cancel cases.

Input (JSON)

Field	Type	Required	Description
`student_identifier`	string	yes	Registration number: two digits, then `U` or `X`, then four or five digits, then an optional `/PS` or `/EVE`. Letters are read in uppercase. Examples: `23/U/9393`, `23/U/22615`, `23/X/93939/EVE`, `23/u/06517/ps` (stored as `23/U/06517/PS`).
`issue_summary`	string	yes	Short description of the unresolved onboarding issue.
`category`	string	yes	One of: `it_webmail_wifi`, `registration`, `fees_informational`, `accommodation`, `other_unclassified`.

The orchestration layer supplies the application **`session_id`**; the model must not set or override it.

Output (JSON)

Field	Type	Description
`status`	string	`created`, `validation_error`, or `error`.
`case_id`	string	On `created`: locally generated id, e.g. `CASE-2026-000001`.
`session_id`	string	Application session id.
`routed_queue`	string	On `created`: queue label (e.g. `ict_helpdesk`, `registration_office`).
`created_at`	string	ISO-8601 UTC timestamp on `created`.
`validation_errors`	array	On `validation_error`: `{ field, message }` entries.
`message`	string	On `error`: human-readable reason.

Permissions

- May **append** one record to `data/cases/drafts.json`.
- May **read** that same file only to allocate the next `case_id` (no other students' details are returned to the model in tool results beyond what the current draft creation requires).
- No network send, no delete, no update of existing cases, no status other than a new draft row.

On failure

- Invalid or missing required fields -> `validation_error` (no write).
- Blocked topic in `issue_summary`, or fee **decision** language when category is fees-related -> `error` (no write).
- Tool not allowed (student has not agreed, or session limit reached) → orchestration returns `error` without calling storage.
- Disk or permission failure -> `error` and log.

Category → queue mapping (deterministic)

Category	Queue label
`it_webmail_wifi`	`ict_helpdesk`
`registration`	`registration_office`
`fees_informational`	`fees_information`
`accommodation`	`accommodation_office`
`other_unclassified`	`general_onboarding`

3. Agent loop (bounded workflow)

Each **student turn** (one message submitted in the UI) runs this loop. The orchestration layer implements **Sense -> Plan/Decide -> Act -> Observe -> Stop or re-plan**.

Phase	What happens
Sense	Load `conversation_history`, session flags (`case_agreed`, `active_case_id`), and the new student message. Run deterministic checks (blocked topic, non-onboarding patterns, prompt-injection heuristics where implemented).
Plan / decide	The model chooses among **approved next actions only**: call `retrieve_documents`, ask the student for missing case fields, ask the student to use the agreement control, produce a final grounded reply, refuse out-of-scope, or stop. It must not skip retrieval before stating onboarding facts.
Act	Orchestration executes at most one batch of tool calls allowed by Section 4, then returns results to the model.
Observe	Tool results (`ok` / `no_match` / `validation_error` / `error`) are appended to the turn trace and to `tool_call_log`.
Stop or re-plan	If a stop condition (Section 5) is met, return the student-facing message. Otherwise continue until a stop condition is met or the iteration limit is reached.

Default policy for onboarding questions

1. If the message is blocked (admissions, grading, disciplinary, fee **decision**) -> refuse; **no tools**.
2. Else -> call `retrieve_documents` at least once before giving factual onboarding content.
3. If `ok` -> answer using returned chunks only; cite `source_title` / URL.
4. If `no_match` or the student needs staff help -> explain honestly; offer a support draft; do **not** call `create_support_case` until Section 4 gate is satisfied.
5. If the student agrees via the application control -> call `create_support_case` with valid fields; on `validation_error`, ask once for corrections and allow **one** retry.
6. Stop when Section 5 applies.

4. State

All state below is **session-scoped** (Streamlit session / application `session_id`) unless noted. It is cleared when the session ends.

State item	Description
`session_id`	UUID assigned by the application; ties drafts to a chat session.
`conversation_history`	Ordered user and assistant messages for multi-turn context.
`case_agreed`	Boolean. **True** only after the student uses the sidebar control **"I agree to save a support draft."** Free-text ("yes", "go ahead") does **not** set this flag.
`active_case_id`	`case_id` of the draft created in this session, if any. Used for follow-ups ("what happened to my case?").
`retrieved_context`	Chunks from the latest successful `retrieve_documents` in this turn (or last turn), for grounding.
`iteration_count`	Number of **model orchestration cycles** in the current student turn (incremented once per model call in the loop).
`tool_call_log`	Ordered list of `{ timestamp, tool, arguments, result }` for audit and Week 5/7 traces.

Persistent memory across browser sessions is **out of scope for Week 5**; planned for Week 6 per project charter.

5. Limits

Enforced by the **orchestration layer**, not by trusting the model.

Limit	Value
Orchestration cycles per student turn	**Maximum 3** model rounds (`MAX_ROUNDS` in `src/tools/orchestrate.py`; may be raised only with documented change and re-test).
Tool calls per student turn	**Maximum 3** total across the turn, enough for a search, a failed case call, and one correction. `create_support_case` itself may run at most twice in that turn: the first attempt and one correction.
Support drafts per session	**Maximum 1**. A second request -> refer the student to `active_case_id`; do not create another draft.
Retrieval breadth	`top_k` ≤ 3; similarity ≥ **0.70**.
Case creation gate	`create_support_case` runs only if `case_agreed == true` and session case count is 0 and the question is not a blocked topic.

Scope limits-the agent must never

- Make admissions, grading, disciplinary, or fee **decisions** (waive, refund, approve fee, etc.).
- State specific dates, fee amounts, or policies **not** present in retrieved chunks.
- Serve a student who says they are a continuing, returning, or postgraduate student.
- Invoke tools other than the two declared tools, or execute resolve/close/reassign actions.
- Transmit a case to external systems (email, SMS, live ticket API)-draft file only.
- Treat prompt-injection text as overriding this contract or as consent to create a case.

6. Stop conditions

The turn **ends** and the UI shows a final assistant message when **any** of the following is true:

ID	Condition	Outcome
S1	**Grounded answer**	`retrieve_documents` returned `ok`; reply cites at least one source; no further tools required.
S2	**Case drafted**	`case_agreed` true; `create_support_case` returned `created`; student receives `case_id` and queue information; **human staff** handle resolution.

ID	Condition	Outcome
S3	**Unsupported / offer draft**	`no_match`; agent states corpus does not confirm the answer and offers a draft; turn may end awaiting agreement control (no case tool until `case_agreed`).
S4	**Out-of-scope refusal**	The student says they are a continuing, returning, or postgraduate student. Polite refusal, no tools. An ordinary onboarding question is not refused for failing to say “I am a freshman.”
S5	**Blocked topic refusal**	Admissions, grading, disciplinary (and fee **decision** requests); refuse; no case tool.
S6	**Adversarial input**	Instruction override or jailbreak attempt; refuse; **no tool calls** for that turn.
S7	**Iteration limit**	`iteration_count` reaches maximum cycles without S1–S2; return a safe fallback message.
S8	**Tool failure after retry**	Tool returned `error` or `validation_error`; agent may **retry once** with corrected input or a single repeated retrieve; if failure persists, report honestly and stop (no hallucinated case id or document text).

Human hand-off: Every successful `create_support_case` (`status: created`) is a hand-off to staff via the draft store and queue label, not an autonomous resolution.

7. What the agent is not

- Not a decision-maker for admissions, grades, discipline, or fees.
- Not an autonomous actor: case creation requires the **agreement control** and produces **drafts** only.
- Not a general-purpose assistant.
- Not a channel that contacts humans directly (no email/SMS).
- Not a cross-session memory system in Week 5.

8. Week 5 evidence (traces)

Required for Week 5 submission: three execution traces stored under `docs/` or `evidence/` (committed paths agreed by the team; trace JSON may live in `evidence/traces/` if the team updates `.gitignore` for those artefacts).

Trace	Scenario
T1	Successful grounded Q&A: retrieve -> cited answer -> stop (S1).
T2	Escalation: `no_match` or explicit need for staff -> student uses agreement control -> valid `create_support_case` -> `case_id` returned (S2).
T3	**Failure and recovery:** e.g. invalid `student_identifier` -> `validation_error` -> agent asks for correction -> retry -> `created` (S8), **or** retrieve `error` -> one retry -> grounded or honest stop.

Recommended additional checks (for evaluation prep, not a substitute for T1–T3):

- Out-of-scope refusal (S4).
- Adversarial refusal without tools (S6).
- Confirmation that a second case in the same session is blocked and that tool calls per turn ≤ 3, with at most one correction of `create_support_case`.

9. Alignment with implementation

This contract matches the tool handlers and case store in:

- `src/tools/handlers.py`
- `src/tools/case_store.py`

- `src/tools/orchestrate.py`
- `src/app.py` (prompt versions `v1.3`, `case_agreed` gate)

Week 5 work extends orchestration into the explicit loop in Section 3, enforces session case limit and per-turn tool cap if not already present, and produces traces T1-T3.